

Sanitized Work Sample: Persistent Enterprise Reasoning Engine

Context

This is a sanitized architecture sample for a system that ingests evolving enterprise documents and maintains a durable, source-backed model of an organization over time.

The goal is not to summarize documents. The goal is to update an enterprise memory: what the company says it is doing, what capabilities it appears to have, what obligations and risks are active, what decisions management has made, and how those items change across filings, transcripts, shareholder letters, and news.

Core Design Principle

A document summary answers:

What did this document say?

A persistent reasoning system answers:

What changed about the enterprise after this document arrived, and what evidence supports that change?

That difference drives the architecture. The document is treated as an event against an existing enterprise state, not as an isolated unit of text.

Prototype Architecture

1. Document Intake

Inputs:

- SEC 10-K, 10-Q, and 8-K filings
- Shareholder letters
- Annual reports
- Earnings call transcripts
- HTML and PDF sources
- News articles in later phases

Pipeline:

1. Fetch document and normalize metadata.

2. Extract text from HTML/PDF.
3. Split into stable sections.
4. Store raw source, normalized text, source URL, company, period, filing type, accession/document id, and retrieval timestamp.
5. Create deterministic section ids so future extraction can be traced back to exact source spans.

2. Structured Extraction

LLMs are used for extraction, not final judgment.

Each section is processed into typed objects such as:

- doctrine
- capability
- active state
- active obligation
- risk
- management decision
- causal relationship
- trajectory signal

Every extracted object must include:

- object type
- normalized label
- evidence quote or source span
- confidence
- time period
- source document id
- whether it is new, continuing, intensified, reduced, contradicted, or resolved

The model is instructed not to invent facts. If evidence is weak, it emits uncertainty rather than filling the gap.

3. Persistent Enterprise State

The state model should be append-friendly and temporal. Current state is materialized from versioned facts rather than overwritten blindly.

Suggested relational core:

```
companies (  
  id,
```

```

    ticker,
    name
)

source_documents (
    id,
    company_id,
    source_type,
    fiscal_period,
    published_at,
    source_url,
    raw_hash,
    ingested_at
)

document_sections (
    id,
    document_id,
    section_path,
    text_hash,
    text
)

enterprise_objects (
    id,
    company_id,
    object_type,
    canonical_label,
    current_status,
    first_seen_at,
    last_seen_at
)

object_versions (
    id,
    object_id,
    document_id,
    section_id,
    period,
    change_type,
    extracted_claim,
    evidence_text,
    confidence,
    created_at
)

causal_edges (
    id,
    company_id,
    source_object_id,
    target_object_id,
    relationship_type,
    evidence_text,
    document_id,
    confidence
)

state_snapshots (
    id,
    company_id,
    as_of_period,
    snapshot_json,
    created_at
)

```

The important part is that the enterprise model has continuity. A risk can persist, worsen, resolve, or be contradicted. A capability can appear, expand, or become strategically less important. A management decision can create future obligations or risks.

4. State Update Logic

For each new document:

1. Retrieve the previous enterprise snapshot.
2. Extract candidate objects and claims from the new source.
3. Match candidates against existing enterprise objects.
4. Decide whether each item is new, continued, changed, contradicted, or resolved.
5. Store new object versions with source evidence.
6. Recompute the current materialized enterprise state.
7. Generate a change log showing what changed and why.

This makes the system useful for longitudinal analysis instead of one-off summaries.

Example State Update

New annual report says:

We are expanding cloud infrastructure capacity to support AI workloads while managing energy and supply chain constraints.

The system should not only summarize that sentence. It should update multiple state objects:

- capability: cloud infrastructure capacity
- active state: AI workload demand increasing
- risk: energy availability / supply chain constraints
- management decision: continued infrastructure investment
- causal edge: AI demand drives infrastructure investment

If the prior year already mentioned infrastructure expansion, the new item is not simply "new." It may be "continued" or "intensified" depending on evidence.

Validation Approach

I would not trust the first version without strict review tooling.

Useful checks:

- every object has source evidence
- every state change has a previous state comparison
- no extracted object without a document/section pointer
- no unsupported financial or strategic claim
- sampling review by company, period, and object type

- deterministic regression fixtures for Microsoft and Amazon annual reports

Prototype Milestone

For a first milestone using Microsoft and Amazon annual reports:

1. Ingest two or more years of annual reports for each company.
2. Parse PDF/HTML into sections.
3. Extract typed enterprise objects.
4. Store source-backed versions in PostgreSQL.
5. Build a state update pass that compares current year to previous year.
6. Produce a structured enterprise state and change log.

The first deliverable does not need a polished UI. A FastAPI service, CLI ingestion runner, PostgreSQL schema, extraction prompts, fixtures, and JSON outputs are enough to validate the architecture.

Relevant Prior Work, Sanitized

- Built production AI and automation systems for high-volume operational workflows where model output had to feed structured downstream processes, not just produce text.
- Built Python/FastAPI data systems with PostgreSQL-backed state, ingestion jobs, normalization logic, and user-facing reporting.
- Built ML/data pipelines that maintain current and historical state across changing external data sources.
- Built AI workflow prototypes where outputs remain review-first, source-backed, and structured for later automation.

Engineering Bias

The risk in this project is not calling an LLM. The risk is letting extracted text become untraceable prose.

I would optimize the first version for:

- source grounding
- temporal state
- simple inspectable schemas
- deterministic ingestion fixtures
- conservative extraction
- clear state diffs

Once those are reliable, graph traversal, embeddings, agent workflows, and deeper reasoning layers become much safer to add.